

Data Science

Feature Engineering

Exploratory Data Analysis



GENERAL ASSEMBLY

Learning objectives



1

Explain the reasons for applying feature engineering to data when modeling.

2

Identify relevant methods to apply to categorical data.

3

Understand how and when to group continuous data into bins.

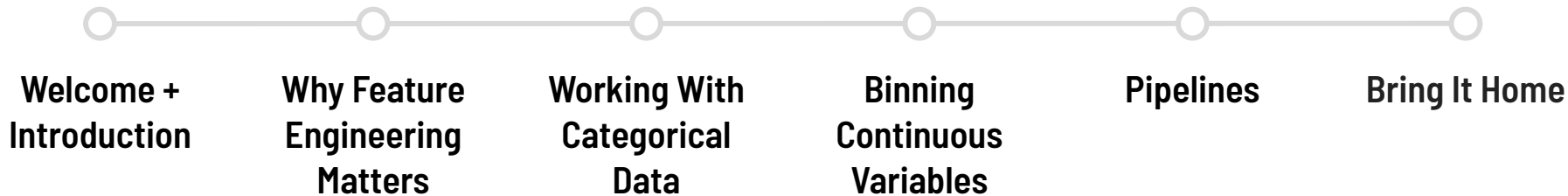
4

Explain the reasons to transform your continuous data when modeling.

5

Create transformers with **scikit-learn** to develop a pipeline of transformations.

Lesson roadmap



Why Feature Engineering Matters

What *is* Feature Engineering?

Feature engineering is the process of **transforming raw data into features that better represent the underlying problem to predictive models.**

It is like preparing ingredients before cooking: you wouldn't throw whole vegetables into soup without chopping them first.

Raw data often needs preparation before it's ready for machine learning.



Why Feature Engineering Matters

Feature engineering is the key to unlocking your data.

Studies show that feature engineering:

- a. Can improve model performance by 10-50%.
- b. Often more impactful than choosing a 'better' algorithm.
- c. Can turn a mediocre model into an excellent one.
- d. Is essential for real-world machine learning success.



Key Considerations in Feature Engineering

1. **Understand your data**: explore and analyse.
2. **Identify data types**: categorical vs continuous.
3. **Handle missing values**: imputation strategies.
4. **Transform features**: scaling, encoding, binning.
5. **Create new features**: domain knowledge.
6. **Validate transformations**: check distributions and relationships.

Different types of data require different approaches

Categorical data:

- a. Ordinal: natural order exists (ratings, education levels)
- b. Nominal: no natural order (colours, cities, brands)

Numerical data:

- a. Discrete: count data (number of bedrooms, clicks)
- b. Continuous: measured data (height, temperature, price)

Common Feature Engineering Challenges

Scale Differences

Some features in the thousands, others in the decimals.

Mixed Data Types

Numbers and categories together.

Skewed
Distributions

Most algorithms prefer normal distributions.

Missing Values

Real data is often incomplete.

Categorical
Explosion

Too many categories can overwhelm models.



Working with Categorical Data

The Challenge

Challenge: Most ML algorithms only understand numbers.

Solution: convert categories to numerical representations.

But how we do this conversion matters for model performance! We will look at 2 approaches here:

1. Label Encoding
2. One-Hot Encoding

Identifying Categorical Variables

1

Text Values

Usually < 50
unique values.

2

Limited Distinct Values

Usually < 50
unique values.

3

Meaningful Groupings

Geographic
regions, product
types.

4

Boolean Flags

Yes/No,
True/False

Numbers can be categorical (ZIP codes, product IDs).

Label Encoding

Concept: assign each category a unique number.

"Poor"	0
"Fair"	1
"Good"	2
"Excellent"	3

When to use: ordinal data with natural ordering.

Label Encoding: Pros and Cons



Pros:

- Simple and memory efficient
- Preserves ordinal relationships
- What else?



Cons:

- Implies order where none may exist...
- Can mislead algorithms about relationships
- What else?

One-Hot Encoding

Concept: create a binary column for each category.

Original: ["Red", "Blue", "Green"]

Becomes:

- Red: [1, 0, 0]
- Blue: [0, 1, 0]
- Green: [0, 0, 1]

One-Hot Encoding: When to Use

Perfect for:

- a. Nominal data (no natural order)
- b. Small number of categories (< 20)
- c. When category relationships matter

Examples:

- a. Geographic regions
- b. Product categories
- c. User types



Label and One-Hot Encoding

Let's see it for ourselves!

- Open **02-feature-engineering.ipynb**



One-Hot Encoding: Pros and Cons

PROS

- a. No false ordinal relationships.
- b. Works with any ML algorithm.
- c. Interpretable results.

CONS

- a. Creates many columns (curse of dimensionality).
- b. Sparse data (mostly zeros).
- c. Memory intensive with many categories.

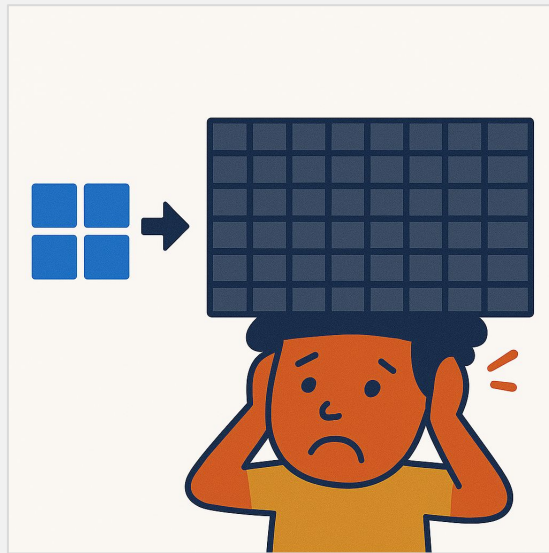
The Curse of Dimensionality

Too many features can hurt performance.

Example:

- a. City variable with 1000 cities becomes 999 new columns with one-hot encoding!
- b. Original dataset: 10 features; after encoding: 1008 features!

Solutions: feature selection, dimensionality reduction, target encoding.



Handling High Cardinality Categories: Strategies

1

Group Rare Categories

'Other' bucket.

2

Target Encoding

Use target variable statistics.

3

Frequency Encoding

Replace with category frequency.

4

Feature Selection

Remove less important categories.

Categorical Encoding: Best Practices

1

Know your data: understand if order matters.

2

Consider cardinality: too many categories? Group them.

3

Domain expertise: employ business knowledge.

4

Test both approaches: sometimes label encoding works better.

5

Watch for leakage: don't use future information.

Binning Continuous Variables

The Reasons for Binning

The Goal: Convert continuous variables into categorical bins.



Capture non-linear relationships.



Reduce impact of outliers.



Make models more interpretable.



Handle missing values naturally.

When to Bin Continuous Variables

Good candidates:

- Age → age groups (young, middle-aged, senior).
- Income → income brackets (low, medium, high).
- Prices → price ranges (budget, mid-range, premium).

Consider binning when:

- You suspect non-linear relationships.
- You have domain knowledge about natural breakpoints.
- You want more interpretable models.

We will focus on two common ways to bin data

- **Equal-Width Binning** divides a data set's range into bins of the same size.
- **Equal-Frequency Binning** divides the data into bins with a similar number of observations in each. (This is also known as **quantile-based** binning.)

Equal-Frequency Binning with `pandas.qcut()`

Concept: each bin contains the same number of observations.

Useful when you want balanced groups:

```
# Create quartiles (4 equal-sized groups)
```

```
df['income_quartile'] = pd.qcut(df['income'],  
                                q=4,  
                                labels=['Q1', 'Q2', 'Q3', 'Q4'])
```

Equal-Width and Equal-Frequency Binning

We can do **Equal-Width Binning** with `pandas.cut()`, and we can do **Equal-Frequency Binning** with `pandas.qcut()`

Let's see it for ourselves!

- Open `02-feature-engineering.ipynb`



Choosing the Right Binning Strategy

EQUAL WIDTH (UNIFORM)	EQUAL FREQUENCY (QUANTILE)	K-MEANS
When data is normally distributed. When you have domain knowledge about ranges.	When data is skewed. When you want balanced groups.	When you want bins based on data clusters. For complex, multi-modal distributions.

K-means is an unsupervised (clustering) modeling approach that is beyond the scope of this lesson

Binning Best Practices

Don't over bin: too many bins = too complex

Don't under-bin: too few bins = lost information

Use domain knowledge: natural breakpoints often work best

Check distributions: ensure bins make sense

Consider the target: how does binning affect your target variable?

Transforming Continuous Variables

Transforming Continuous Variables

The challenge: many ML algorithms work better when:

- Features are on similar scales
- Distributions are approximately normal
- No extreme outliers exist

The solution: mathematical transformations.

Why transform continuous variables?

Scale Issues:

- Income: \$30,000-\$200,000
- Age: 18-80
- Square footage: 500-5000

Without scaling, income dominates the model!

Algorithm sensitivity: KNN, SVM, Neural Networks: very sensitive to scale.

Tree-based models: less sensitive.

Types of Transformations of Continuous Variables

Scaling transformations:

- StandardScaler (Z-score normalisation)
- MinMaxScaler (0-1 scaling)
- RobustScaler (median-based)

Distribution transformations:

- Log transformation
- Power transformation (Box-Cox, Yeo-Johnson)
- Quantile transformations

Choosing the Right **Scaling** Strategy

StandardScaler	MinMaxScaler	RobustScaler
Data is approximately normal.	Data has hard min/max bounds.	Data has outliers.
You don't need specific bounds.	You need values in specific range.	Uses median instead of mean, and scales based on the IQR.
Most common choice.	Sparse data.	For outliers.

Choosing the Right **Scaling** Strategy

StandardScaler	MinMaxScaler	RobustScaler
Z-score Normalisation. Mean = 0, StDev = 1	Rescale to range [0, 1] or [-1, 1]	Uses median and IQR as reference points.
$z = \frac{x - \mu}{\sigma}$	$z = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$	$x' = \frac{x - \text{median}(x)}{\text{IQR}(x)}$

Distribution Transformations: Why?

Many algorithms assume normal distributions

Common problems:

- Right-skewed data (income, prices)
- Left-skewed data (grades, ratings)
- Multi-modal distributions

Solution: transform to make the data more normal.

Log Transformation

Best for: right-skewed data

```
df['log_income'] = np.log(df['income'])
```

Effect:

- Compresses large values
- Expands small values
- Often makes right-skewed data more normal

Only works with positive values though.

Power Transformations

Box-Cox: for positive data only

Yeo-Johnson works with negative values too.

Automatically finds the best.

```
from sklearn.preprocessing import PowerTransformer
```

```
# Box-Cox pt = PowerTransformer(method='box-cox')
```

```
transformed = pt.fit_transform(df[['positive_feature']])
```

```
# Yeo-Johnson pt = PowerTransformer(method='yeo-johnson')
```

```
transformed = pt.fit_transform(df[['any_feature']])
```

Quantile Transformations

Concept: map to uniform or normal distribution.

Powerful, but removes original relationships.

```
from sklearn.preprocessing import QuantileTransformer
```

```
# Map to uniform distribution
```

```
qt = QuantileTransformer(output_distribution='uniform')
```

```
# Map to normal distribution
```

```
qt = QuantileTransformer(output_distribution='normal')
```

```
transformed = qt.fit_transform(df[['feature']])
```

Choosing the Right Transformation

1

Log

Right-skewed data; want interpretability.

2

Power

Want automatic optimal transformation.

3

Quantile

Want specific distribution shape.

4

Standard/Min/Max

Just need scaling.

Distribution Transformations

Scaling Transformations

Transforming Continuous Variables

Let's see it for ourselves!

- ☐ Open **02-feature-engineering.ipynb**
- ☐ Go to the **Transforming Continuous Variables** section



Pipelines

What Pipelines Are

A **pipeline** = a sequence of data transformations followed by a model.

Benefits:

- Prevents data leakage: fit only on training data
- Reproducible: same transformations every time
- Clean code: all preprocessing in one place
- Easy deployment: transform new data identically

Data Leakage: The Silent Killer

What is it? Using future information to make predictions.

```
# WRONG - fits scaler on all data
```

```
scaler.fit(df[['feature']])
```

```
X_train_scaled = scaler.transform(X_train)
```

```
# RIGHT - fits scaler only on training data
```

```
scaler.fit(X_train[['feature']])
```

```
X_train_scaled = scaler.transform(X_train)
```

Result if done incorrectly: Overly optimistic performance estimates.

Basic Pipeline Structure

```
from sklearn.pipeline import Pipeline
```

```
pipeline = Pipeline([ ('preprocessing_step', SomeTransformer()), ('model_step',  
SomeModel()) ])
```

```
# Fit and predict in one go
```

```
pipeline.fit(X_train, y_train) predictions = pipeline.predict(X_test)
```

Key: each step transforms data for the next step.

ColumnTransformer

Different columns have different treatments.

Problem: numeric and categorical columns need different preprocessing.

Result: apply different transformations to different columns.

```
from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer([ ('num', StandardScaler(), numeric_features),
                                   ('cat', OneHotEncoder(), categorical_features) ])
```

Pipelines

Let's make a complete pipeline in Python.

- ☐ Open **02-feature-engineering.ipynb**
- ☐ Go to the **Building Feature Engineering Pipelines** section



Pipeline: Best Practices

- 1 Always split data first**: before any transformations.
- 2 Fit on training only**: never fit transformers on test data.
- 3 Use cross-validation**: for honest performance estimates.
- 4 Save your pipelines**: for deployment and reproducibility.
- 5 Document transformations**: know what each step does.

Custom Transformers

When you need transformations, but not in scikit-learn:

```
from sklearn.base import BaseEstimator, TransformerMixin

class AgeGroupTransformer(BaseEstimator, TransformerMixin):

    def fit(self, X, y=None):

        return self

    def transform(self, X):

        X = X.copy()

        X['age_group'] = pd.cut(X['age'], bins=[0, 18, 35, 65, 100],
                                labels=['Child', 'Adult', 'Middle', 'Senior']) return X
```



Handle Missing Values in Pipelines

```
from sklearn.impute import SimpleImputer
```

```
# Numeric pipeline with imputation
```

```
numeric_pipeline = Pipeline([ ('imputer', SimpleImputer(strategy='median')),  
                               ('scaler', StandardScaler()) ])
```

```
# Categorical pipeline with imputation
```

```
categorical_pipeline = Pipeline([ ('imputer',  
SimpleImputer(strategy='most_frequent')), ('encoder', OneHotEncoder(drop='first')) ])
```

Bring It Home

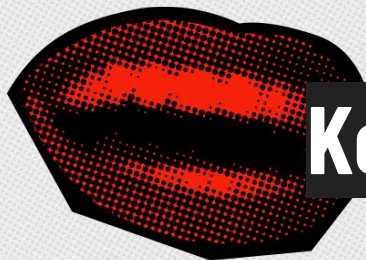
Feature Engineering Strategy

1. **Start simple**: basic scaling and encoding.
2. **Understand your data**: plot distributions, check relationships
3. **Apply domain knowledge**: what makes business sense?
4. **Iterate**: try different transformations.
5. **Measure impact**: always validate improvements.
6. **Document everything**: for reproducibility.



Common Pitfalls to Avoid

1. **Data leakage**: fitting transformers on test data.
2. **Forgetting to scale**: mixing different scales.
3. **Over-engineering**: too many complex transformations.
4. **Ignoring domain knowledge**: data science isn't just maths.
5. **Not validating**: always check your transformations worked.



Key takeaways



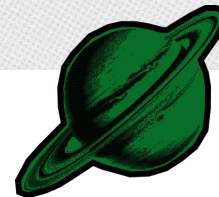
Feature Engineering is important

It can make a big difference to how good your model is



There are lots of techniques you can use

Use a strategy to get the best results



Be aware of common pitfalls

Including data leakage and domain expertise

